

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



C PROGRAMMING ASSIGNMENT

Student Name	SUDHIR MADHAVAN K
Register Number	1925260205
Course Code & Name	CSA0211
Slot	D
Assignment / Task Title	**Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative**
GitHub Repository URL	https://github.com/sudhirmadhavan/CSA0211-C-PROGRAMING.git
Submission Date	27.10.2026

TASK 1 – Organise the Program with Header Files and Conditional Compilation (3 Marks)

1. Aim

To organise the electricity billing program by placing shared constants and function prototypes in a header file, using include guards, and adding a compile-time debugging mode controlled by #ifdef DEBUG.

2. Concept / Theory

A C program is commonly organised into header files, global declarations, the main() function, and user-defined functions. Header files share declarations between source files. Global declarations include symbolic constants and function prototypes. main() coordinates input, calculation, and output, while other functions perform focused operations.

2.1 Include Guards

Include guards prevent a header from being processed repeatedly in the same translation unit. #ifndef checks whether a macro is undefined, #define defines it, and #endif closes the conditional block.

```
#ifndef BILLING_H
#define BILLING_H
/* Shared declarations go here. */
#endif
```

2.2 Conditional Compilation

The #ifdef DEBUG directive includes diagnostic statements only when DEBUG is defined during preprocessing. Compile with gcc -DDEBUG to include the diagnostic messages. Without that option, those statements are removed before compilation.

2.3 Angle-Bracket and Quoted Includes

#include <stdio.h> is used for a standard header. #include "billing.h" is used for a user-created header; the compiler normally searches the local source directory first for a quoted include.

3. Procedure / Algorithm

1. Create billing.h and place tariff constants and function prototypes in it.
2. Protect the header with include guards.
3. Create main.c and include stdio.h and billing.h.
4. Implement consumption calculation with one-rollover support.
5. Implement slab billing using integer paise.
6. Place intermediate diagnostic printf statements inside #ifdef DEBUG blocks.
7. Compile and run once normally and once with -DDEBUG.
8. Compare both outputs and save screenshots.

4. Program / Source Code

File 1: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define FIXED_CHARGE_PAISE 5000LL
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL

unsigned int calculate_units(unsigned short previous,
                             unsigned short current);
long long calculate_bill(unsigned int units);

#endif
```

File 2: main.c

```
#include <stdio.h>
#include "billing.h"

unsigned int calculate_units(unsigned short previous,
                             unsigned short current)
{
    unsigned int units;

    if (current >= previous)
        units = (unsigned int)(current - previous);
    else
        units = (65536U - (unsigned int)previous)
              + (unsigned int)current;

#ifdef DEBUG
    printf("Debug: previous reading = %hu\n", previous);
    printf("Debug: current reading = %hu\n", current);
    printf("Debug: units consumed = %u\n", units);
#endif

    return units;
```

```
}

long long calculate_bill(unsigned int units)
{
    long long bill = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        bill += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
        bill += (long long)(SLAB2_UNITS - SLAB1_UNITS) * RATE2_PAISE;
    } else if (units > SLAB1_UNITS) {
        bill += (long long)(units - SLAB1_UNITS) * RATE2_PAISE;
    }

#ifdef DEBUG
    printf("Debug: bill = %lld paise\n", bill);
#endif

    return bill;
}

int main(void)
{
    unsigned int previous_input, current_input;
    unsigned short previous, current;
    unsigned int units;
    long long bill;

    printf("Enter previous reading (0-65535): ");
    if (scanf("%u", &previous_input) != 1 || previous_input > 65535U) {
        printf("Invalid previous reading.\n");
        return 1;
    }

    printf("Enter current reading (0-65535): ");
    if (scanf("%u", &current_input) != 1 || current_input > 65535U) {
        printf("Invalid current reading.\n");
        return 1;
    }

    previous = (unsigned short)previous_input;
    current = (unsigned short)current_input;
    units = calculate_units(previous, current);
    bill = calculate_bill(units);

    printf("Units consumed: %u\n", units);
    printf("Total bill: Rs %lld.%02lld\n", bill / 100, bill % 100);
    return 0;
}
```

5. Compilation and Output

Compile without debug statements:

```
gcc -std=c11 -Wall -Wextra main.c -o billing
```

Compile with debug statements:

```
gcc -std=c11 -Wall -Wextra -DDEBUG main.c -o billing_debug
```

Example input: previous reading 1000; current reading 1150.

Expected normal output:

```
Enter previous reading (0-65535): 1000
Enter current reading (0-65535): 1150
Units consumed: 150
Total bill: Rs 162.50
```

With -DDEBUG, the program also prints the previous reading, current reading, units consumed, and bill in paise. For readings 65530 and 10, consumption is 16 units.

6. Alternative Concept and Complexity Analysis

Alternative: use a run-time variable such as int debug = 1; and test if (debug) before each diagnostic printf. This lets the program switch debugging on or off while it is running. The test is O(1) per debug statement; for k statements the total check cost is O(k).

Feature	#ifdef DEBUG	Run-time if (debug)
When selected	At compile time	While the program runs
Checks when disabled	No run-time debug test	A condition test remains
Executable size	Debug statements can be excluded	Debug code usually remains
Per statement time	No debug branch when excluded	O(1) test
Auxiliary data space	O(1)	O(1) for the flag

For n households, processing all bills is O(n), because each household requires a fixed number of calculations. Processing one household at a time requires O(1) auxiliary space; storing all household records requires O(n) space.

7. Analysis / Discussion

The header file centralises the tariff constants and prototypes. Include guards protect against repeated inclusion. The DEBUG build helps trace intermediate values, while the normal build avoids diagnostic output. The code validates the reading range and assumes the meter is a 16-bit register with at most one rollover between readings.

8. Result

The program was organised using a header file and include guards. Compile-time debugging was implemented and can be enabled using gcc -DDEBUG. Normal and rollover consumption cases can be tested with the same billing functions.

TASK 2 – Read and Print Data Correctly (4 Marks)

1. Aim

To use the correct scanf and printf format specifiers, validate input by checking scanf's return value, demonstrate the risk of mismatched format specifiers, and print monetary amounts stored in integer paise.

2. Concept / Theory

Formatted input and output functions require format specifiers that match the data types of the corresponding arguments. A mismatch can cause incorrect output or undefined behaviour, particularly when scanf writes through a pointer of the wrong type.

Data type	scanf specifier	printf specifier	Use
int	%d	%d	Signed integer
unsigned int	%u	%u	Non-negative integer
long	%ld	%ld	Long integer
long long	%lld	%lld	Wide monetary total
float	%f	%f	Single-precision value
double	%lf	%f or %lf	For printf, %f is used for double; %lf is also accepted by modern C implementations
char	%c	%c	Character; leading space skips whitespace
char array	%s	%s	Word string; width limit is important

For scanf, %f expects float * and %lf expects double *. For printf, a float argument is promoted to double, so %f is used to print both float and double values. Use a width such as %49s for a 50-byte character array.

2.1 Detect Invalid Input

scanf returns the number of input items successfully assigned. For two conversions, check that it returns 2. If the user enters letters instead of a number, the conversion fails and the program should report invalid input. In an interactive program, invalid characters may remain in the input stream; line-based input is often easier to recover from.

3. Procedure / Algorithm

9. Declare meter readings and the bill amount using suitable data types.
10. Read each value using the matching format specifier.
11. Check the return value of scanf before using the input.
12. Reject values outside the valid meter-reading range.
13. Calculate consumption and the bill in integer paise.
14. Print the rupees and paise using division and remainder.
15. Test valid numeric input and invalid alphabetic input.

4. Program / Source Code

File: input_output.c

```
#include <stdio.h>

int main(void)
{
   unsigned int previous, current;
   unsigned int units;
   long long bill_paise = 5000LL;
```

```
    printf("Enter previous reading (0-65535): ");
    if (scanf("%u", &previous) != 1 || previous > 65535U) {
        printf("Invalid previous reading.\n");
        return 1;
    }

    printf("Enter current reading (0-65535): ");
    if (scanf("%u", &current) != 1 || current > 65535U) {
        printf("Invalid current reading.\n");
        return 1;
    }

    if (current >= previous)
        units = current - previous;
    else
        units = (65536U - previous) + current;

    if (units > 200U) {
        bill_paise += (long long)(units - 200U) * 450LL;
        bill_paise += 100LL * 225LL;
    } else if (units > 100U) {
        bill_paise += (long long)(units - 100U) * 225LL;
    }

    printf("Units consumed: %u\n", units);
    printf("Bill: Rs %lld.%02lld\n",
           bill_paise / 100, bill_paise % 100);
    return 0;
}
```

5. Output and Validation

Valid input example: previous = 1000, current = 1150.

```
Enter previous reading (0-65535): 1000
Enter current reading (0-65535): 1150
Units consumed: 150
Bill: Rs 162.50
```

Invalid input example: the user types abc when a number is requested.

```
Enter previous reading (0-65535): abc
Invalid previous reading.
```

5.1 Wrong Format Specifier Test

Example of an incorrect printf call:

```
long value = 123456789L;
printf("%d\n", value);   /* Incorrect: %d expects int */
```

The format does not match the argument type. The behaviour is undefined by the C standard, so the output cannot be relied upon. Compile with gcc -Wall -Wextra to receive a format warning. The correction is printf("%ld\n", value);. Similarly, scanf must use the correct pointer type, such as scanf("%ld", &value) for long.

5.2 Printing Rupees and Paise

If bill_paise is 16250, bill_paise / 100 gives 162 rupees and bill_paise % 100 gives 50 paise. The format %lld.%02lld ensures that paise is printed using two digits.

6. Alternative Concept and Complexity Analysis

Alternative: read a complete line using fgets(), then convert it with strtol(). This allows the program to detect trailing characters and recover by discarding or re-reading the entire line.

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

int main(void)
{
   char line[128];
   char *end;
   long reading;

   printf("Enter a meter reading: ");
   if (fgets(line, sizeof line, stdin) == NULL) {
      puts("Input error.");
      return 1;
   }

   errno = 0;
   reading = strtol(line, &end, 10);

   if (end == line || errno == ERANGE ||
      reading < 0 || reading > 65535) {
      puts("Invalid reading.");
      return 1;
   }

   while (*end == ' ' || *end == '\t' ||
         *end == '\n' || *end == '\r')
      end++;

   if (*end != '\0') {
      puts("Unexpected characters in input.");
      return 1;
   }

   printf("Reading accepted: %ld\n", reading);
   return 0;
}
```

For a line of length L, reading and scanning the line takes O(L) time. A fixed buffer of size B uses O(B) space; here B is 128 bytes. Direct scanf is concise, but recovery from invalid input is less convenient because unconsumed characters may remain in stdin. fgets() plus strtol() gives better control over validation and error recovery.

7. Analysis / Discussion

The program uses format specifiers that match the argument types, checks scanf's return value, validates the meter range, and stores the bill in integer paise. The line-based alternative is more robust for interactive input because it validates a complete line and can distinguish a numeric value from extra unwanted characters.

8. Result

Correct format specifiers and input validation were implemented. Valid readings produce a correctly formatted bill, while invalid input is rejected. The alternative fgets() and strtol() method was analysed for validation, time, and buffer space.

TASK 3 – Control Type Conversion and Precision (4 Marks)

1. Aim

To analyse implicit and explicit type conversions in C, identify integer-division and narrowing-conversion problems, demonstrate unexpected expressions, and compare integer-paise billing with floating-point calculations.

2. Concept / Theory

2.1 Implicit Conversion and Integer Promotion

When arithmetic combines different types, C applies integer promotions and the usual arithmetic conversions. For example, a short integer is commonly promoted to int before arithmetic. When an int is combined with a double, the int is converted to double.

2.2 Integer Division

When both operands are integers, division discards the fractional part toward zero. Therefore, 5 / 2 evaluates to 2, not 2.5. Use 5.0 / 2 or cast one operand to double when a fractional result is required.

2.3 Explicit Casting and Narrowing

An explicit cast requests conversion to another type. If a long value cannot be represented by int, converting it to int is implementation-defined in the relevant C standard rules; the result must not be assumed portable. Validate ranges before narrowing values.

2.4 Monetary Rounding

The billing system stores money as integer paise. When a tariff calculation produces a fractional paisa, the program must apply a documented rule, such as rounding to the nearest paisa with half-paisa values rounded upward for non-negative charges. The supplied slab rates are whole paise per unit, so these example calculations do not require fractional-paisa rounding.

3. Procedure / Algorithm

16. Identify expressions that combine integers, floating-point values, and wider integer types.
17. Compile a demonstration program that prints integer division, narrowing conversion, and floating-point comparison results.
18. Explain why each result may differ from an expectation.
19. Compare repeated float accumulation with integer-paise accumulation.
20. Record the compiler and observed output, because some conversion results depend on the implementation.

4. Program / Source Code

File: conversions.c

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int a = 5, b = 2;
    int truncated = (int)3.9;
    double x = 0.1 + 0.2;
    long large_value = (long)INT_MAX + 1L;
    int narrowed = (int)large_value;

    printf("Integer division 5 / 2 = %d\n", a / b);
```

```
    printf("Conversion (int)3.9 = %d\n", truncated);
    printf("0.1 + 0.2 with 17 digits = %.17f\n", x);
    printf("0.1 + 0.2 equals 0.3? %s\n",
           x == 0.3 ? "yes" : "no");
    printf("INT_MAX = %d\n", INT_MAX);
    printf("Casting INT_MAX + 1L to int gives: %d\n", narrowed);

    return 0;
}
```

5. Output and Explanation

Typical output on a system with 32-bit int and IEEE 754 double:

```
Integer division 5 / 2 = 2
Conversion (int)3.9 = 3
0.1 + 0.2 with 17 digits = 0.30000000000000004
0.1 + 0.2 equals 0.3? no
INT_MAX = 2147483647
Casting INT_MAX + 1L to int gives: implementation-dependent result
```

The last line above is explanatory, not literal program output: the program prints the actual result on the compiler being used. On common GCC systems with 32-bit int, the result is often -2147483648, but this must not be relied upon as portable behaviour.

5.1 Three Unexpected Results

21. 5 / 2 gives 2 because both operands are integers and the fractional part is discarded.
22. (int)3.9 gives 3 because conversion from floating point to integer truncates toward zero; it does not round to the nearest integer.
23. 0.1 + 0.2 may produce 0.30000000000000004, so direct equality with the double literal 0.3 can be false due to binary floating-point representation.

A fourth important case is casting a long value outside the range of int. The resulting value is implementation-defined on common implementations, so a program must check the range before converting.

6. Alternative Concept and Complexity Analysis

Alternative: calculate all charges using float or double rupees and round only once at the end. This is easy to read but can accumulate representation error. The error depends on the values, order of operations, precision, and compiler; it is not guaranteed to grow linearly with the number of bills.

The following program compares repeated float addition with exact integer-paise addition for several values of n.

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    const int counts[] = {10, 100, 1000, 4000};
    const int cases = (int)(sizeof counts / sizeof counts[0]);

    for (int j = 0; j < cases; j++) {
        int n = counts[j];
        float total_float = 0.0f;
        long long total_paise = 0;

        for (int i = 0; i < n; i++) {
            total_float += 0.10f;
            total_paise += 10;
        }
```

```
        double rounded_float = round((double)total_float * 100.0) / 100.0;
        double exact_total = (double)total_paise / 100.0;

        printf("n=%d, float=%.9f, rounded=%.2f, integer=%.2f\n",
               n, total_float, rounded_float, exact_total);
    }
    return 0;
}
```

Compile using: gcc -std=c11 -Wall -Wextra float_comparison.c -o float_comparison -lm

Integer paise gives the exact total for whole-paisa amounts if the integer type does not overflow. float generally occupies 4 bytes, while long long typically occupies 8 bytes. Each addition is O(1), total time for n bills is O(n), and space per running total is O(1).

7. Conversion Points in the Billing Program

- Meter subtraction: unsigned short operands are commonly promoted to int before subtraction; an explicit unsigned conversion or rollover formula makes the intended result clear.
- Slab calculation: units minus a slab limit is integer arithmetic; convert to long long before multiplication to reduce overflow risk.
- Rate multiplication: the unit count and paise rate must be converted to a sufficiently wide integer type before multiplication.
- Currency formatting: integer division by 100 gives rupees, and remainder by 100 gives paise.
- Input conversion: converting unsigned int readings to unsigned short is safe only after validating the range 0–65535.

8. Analysis / Discussion

The examples show that type conversion rules can change results. Integer division discards fractions, floating-point arithmetic is approximate, and narrowing conversions can lose information. For reliable billing, use sufficiently wide integer types for paise and explicitly document any rounding rule required by a tariff.

9. Result

Type-conversion behaviour was demonstrated and analysed. Three unexpected results were explained, and the alternative floating-point approach was compared with integer-paise calculation in terms of precision, time complexity, and storage.

TASK 4 – Trace the Compilation Process (4 Marks)

1. Aim

To trace a C program through preprocessing, compilation, assembly, and linking; distinguish compile-time, link-time, and run-time errors; interpret GCC warnings; and compare a single-file build with a multi-file build.

2. Concept / Theory

2.1 Stages of Compilation

24. Preprocessing: expands macros and processes #include, #define, and conditional-compilation directives.
25. Compilation: translates preprocessed C code into assembly language and reports language errors and warnings.
26. Assembly: converts assembly instructions into an object file containing machine code and symbol information.
27. Linking: combines object files and required libraries to create the final executable.

2.2 GCC Commands

Command	Output	Purpose
gcc -E main.c -o main.i	main.i	Preprocessed source
gcc -S main.c -o main.s	main.s	Assembly source
gcc -c main.c -o main.o	main.o	Object file
gcc main.o -o billing	billing / billing.exe	Linked executable
gcc -std=c11 -Wall -Wextra main.c -o billing	Executable plus diagnostics	Compile with common warnings enabled

2.3 What the Preprocessor Changes

The preprocessor inserts the contents of included headers, replaces macro names with their replacement text, and removes inactive conditional sections. For example, a #define tariff macro is expanded to its value, and code inside #ifdef DEBUG is omitted when DEBUG is not defined.

3. Procedure / Algorithm

28. Save the program as main.c.
29. Run gcc -E -P to create and inspect preprocessed output.
30. Run gcc -S to generate assembly.
31. Run gcc -c to create an object file.
32. Link the object file to produce an executable.
33. Compile with -Wall -Wextra and inspect all warnings.
34. Record examples of compile-time, link-time, and run-time errors.
35. Compare rebuilding one file in a single-file program with rebuilding one file in a multi-file program.

4. Program / Source Code

Use the following small program as the compilation-stage example. Save it as main.c.

```
#include <stdio.h>

#define FIXED_CHARGE_PAISE 5000LL

static long long total_bill(long long energy_paise)
{
   return FIXED_CHARGE_PAISE + energy_paise;
}
```

```
int main(void)
{
    long long bill = total_bill(11250LL);
    printf("Total bill: Rs %lld.%02lld\n", bill / 100, bill % 100);
    return 0;
}
```

5. Compilation Commands and Expected Result

```
gcc -E -P main.c -o main.i
gcc -S main.c -o main.s
gcc -c main.c -o main.o
gcc main.o -o billing
./billing
```

On Windows, run billing.exe instead of ./billing.

```
Total bill: Rs 162.50
```

6. Error and Warning Classification

Category	Example	Explanation / Fix
Compile-time error	Missing semicolon after a declaration	The compiler cannot parse the source; add the semicolon.
Link-time error	Call a declared function whose definition is not linked	The compiler may accept the call, but the linker cannot find the function; compile/link the file that defines it.
Run-time error	Attempt to use an invalid array index or divide by zero	The program has started but fails or behaves incorrectly during execution; validate values before the operation.
Compiler warning	printf("%d", a_long_value)	The format does not match the type; use %ld for long or %lld for long long.

A format mismatch in printf can produce undefined behaviour. Do not treat a particular wrong output as guaranteed. The correct response is to fix the format and compile with -Wall -Wextra.

7. Alternative Concept and Complexity Analysis

Alternative: split the program into input.c, tariff.c, report.c, and a shared billing.h. Each source file includes the shared header and is compiled separately. The linker then combines the object files.

Example build commands:

```
gcc -std=c11 -Wall -Wextra -c input.c -o input.o
gcc -std=c11 -Wall -Wextra -c tariff.c -o tariff.o
gcc -std=c11 -Wall -Wextra -c report.c -o report.o
gcc -std=c11 -Wall -Wextra -c main.c -o main.o
gcc input.o tariff.o report.o main.o -o billing
```

Build situation	Single source file	Separate source files
Initial build	Compile and link the one source	Compile each source, then link all objects
Change one function	Usually recompile the entire source file	Usually recompile only the changed source, then relink
Compile work after a small change	O(S) for source size S	O(S_changed) plus linking, assuming dependencies are tracked
Object-file storage	One object file	One object file per source file
Maintenance	Simple for small programs	Better separation for larger projects

If a shared header changes, every source file that depends on it generally needs recompilation. Object-file storage is proportional to the generated machine code and symbol/debug information; the exact byte count depends on compiler options and platform. The number of object files grows with the number of source files, while total build work is approximately proportional to the sum of compiled source sizes plus linking work.

8. Analysis / Discussion

The compilation commands expose each stage of the build. Preprocessing shows expanded headers and macros; assembly and object files are intermediate artifacts; linking creates the executable. Compiler warnings are useful evidence of type and format mistakes. Separating the program into multiple source files can reduce recompilation work when only one source file changes, provided the build dependencies are managed correctly.

9. Result

The preprocessing, compilation, assembly, and linking stages were identified with the corresponding GCC commands. Compile-time, link-time, and run-time errors were distinguished, and the benefits of a multi-file build were compared with a single-file build.

SUBMISSION CHECKLIST

- Use the repository name CSA02-<RegisterNumber>-Assignment1.
- Keep a separate folder for every task and include source code and input files where applicable.
- Run every program using GCC and save genuine output screenshots.
- Include the exact GCC version and compilation commands in README.md.
- Commit work progressively, with at least one commit per task.
- Add the working GitHub repository link to the cover page and submission form.
- Before submitting, replace any sample name/register number if your actual details differ.

README.md – Suggested Content

```
# CSA02-<RegisterNumber>-Assignment1

Student Name: SUDHIR MADHAVAN K
Register Number: 1925260205
Course: CSA0211 – C Programming
Compiler: GCC (write the version shown by `gcc --version`)
Operating System: (write your OS)

## Task 1
Compile normal build:
gcc -std=c11 -Wall -Wextra main.c -o billing

Compile debug build:
gcc -std=c11 -Wall -Wextra -DDEBUG main.c -o billing_debug

## Task 2
gcc -std=c11 -Wall -Wextra input_output.c -o input_output

## Task 3
gcc -std=c11 -Wall -Wextra conversions.c -o conversions
gcc -std=c11 -Wall -Wextra float_comparison.c -o float_comparison -lm

## Task 4
gcc -E -P main.c -o main.i
gcc -S main.c -o main.s
gcc -c main.c -o main.o
gcc main.o -o billing

Add a short result note and the output screenshot for each task.
```

Final reminder: update the sample identity and GitHub URL if you are submitting under different details. Capture your own compiler output; screenshots in the report should reflect programs you actually executed.